Guía de ejercicios 3 — Patrones de comportamiento

Programación II — TSSI · Unidad 4

Indicaciones

Los patrones de comportamiento resuelven cómo se reparten las responsabilidades y cómo se comunican los objetos entre sí. Esta guía tiene un ejercicio por patrón.

- Resolver cada ejercicio en una aplicación de consola C# (`dotnet new console`).
- Los enunciados no dicen qué clases crear: decidir las clases, interfaces y relaciones es parte del ejercicio. Antes de programar, dibujar el diagrama de clases.
- Programar contra interfaces o clases abstractas; el `Main` solo arma objetos y muestra resultados.
- Una clase por archivo, con nombres en inglés como en el material de lectura (pág. 81-84).
- Comparar lo que muestra tu programa con la salida esperada y responder las preguntas de "Para pensar".

Ejercicios principales

Ejercicio 1 — Strategy: costo de envío

Una tienda online ofrece tres formas de envío: estándar ($1000 por kg), express ($1800 por kg) y retiro en local (sin cargo). En el checkout, el cliente puede cambiar de opción todas las veces que quiera antes de pagar.

El área comercial avisó que cada mes habrá promociones de envío con reglas nuevas. El pedido tiene que calcular su costo de envío sin `if` ni `switch` según el tipo de envío, y sumar una forma de envío no debe obligar a modificar el pedido.

Para verificar: con un pedido de 3 kg, cambiar la forma de envío tres veces y mostrar el costo en cada caso.

```
Estándar: $3000
Express: $5400
Retiro en local: $0
```

Para pensar: ¿qué principio SOLID se rompería si el cálculo estuviera dentro del pedido con un `switch`? ¿Cómo sumarías una promo de "envío gratis desde 5 kg"?

Ejercicio 2 — Observer: novedades de la cátedra

La cátedra tiene un canal de novedades. Cada alumno puede suscribirse para recibir las novedades por email o por la app, y darse de baja cuando quiera. Cada vez que se publica una novedad, la reciben todos los suscriptos en ese momento.

El canal no debe conocer los medios de aviso concretos, porque más adelante se puede sumar WhatsApp.

Para verificar: suscribir a Ana (email) y a Luis (app), publicar una novedad, dar de baja a Luis y publicar otra.

```
Ana recibió por email: Se suspenden las clases del viernes
Luis recibió en la app: Se suspenden las clases del viernes
Ana recibió por email: Ya están las notas del parcial
```

Para pensar: ¿qué tendrías que tocar para sumar WhatsApp? ¿Qué pasaría si un suscriptor se diera de baja justo mientras recibe una novedad?

Ejercicios complementarios

Ejercicio 3 — Chain of Responsibility: aprobación de gastos

En una empresa, cada gasto lo aprueba el primer nivel con autoridad suficiente: el supervisor hasta $50.000, el gerente hasta $200.000 y el director sin límite. Quien carga un gasto siempre se lo presenta al supervisor y no sabe quién lo va a terminar aprobando.

La empresa planea sumar un nivel "jefe de área" entre el supervisor y el gerente. Ese cambio debe resolverse agregando el nivel nuevo y armando la secuencia de otra forma, sin modificar los niveles existentes.

Para verificar: presentar al supervisor gastos de $30.000, $120.000 y $500.000.

```
$30000 aprobado por Supervisor
$120000 aprobado por Gerente
$500000 aprobado por Director
```

Para pensar: ¿qué pasa si ningún nivel puede atender un gasto? Proponer una forma de avisarlo.

Ejercicio 4 — Template Method: máquina de bebidas

Una máquina expendedora prepara té y café. Las dos bebidas siguen la misma secuencia: hervir agua, preparar la infusión, servir en la taza y agregar extras. Hervir y servir son idénticos para todas; preparar y agregar extras cambian (té: saquito y limón; café: filtrado, azúcar y leche).

El fabricante va a sumar bebidas, como el mate cocido. Ninguna bebida nueva debe poder alterar el orden de la secuencia ni repetir el código de los pasos comunes.

Para verificar: preparar un té y un café.

```
-- Té --
Hirviendo agua
Remojando el saquito
Sirviendo en la taza
Agregando limón
```

```
-- Café --
Hirviendo agua
Filtrando el café
Sirviendo en la taza
Agregando azúcar y leche
```

Para pensar: ¿cómo garantiza tu solución que el orden no cambie? Comparar con Strategy: ¿cuál usa herencia y cuál composición?